UNIT-4 Immutable data structure

Immutable data types are those that can never change their value in place.

Example: String, Tuple etc.

String

Any sequence of characters within either single quotes or double quotes is considered as a String.

In [8]:
```
s = 'Arman'
s = "Dhairya"
```

We can define multi-line String literals by using triple single or double quotes.

In [2]:
```
s ='''Bharat
       Gas'''
```

Accessing Characters Of String:

We can access characters of a string by using the following ways.

(1) By using index

(2) By using slice operator

(1) Accessing Characters By using Index:-

Python supports both +ve and -ve Index.

+ve Index means Left to Right (Forward Direction)

-ve Index means Right to Left (Backward Direction)

In [3]:
```
s = "Aradhya"
print(s[4])
print(s[-1])
print(s[0])
print(s[9])
```

```
h
a
A

---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
<ipython-input-3-02a41c24ee85> in <module>
      3 print(s[-1])
      4 print(s[0])
----> 5 print(s[9])

IndexError: string index out of range
```

(2) Accessing Characters by using Slice Operator:-

Syntax: s[beginindex:endindex:step]

Begin Index: From where we have to consider slice.

End Index: We have to terminate the slice at endindex-1

Step: Incremented Value.

If we are not specifying begin index then it will consider from beginning of the string. If we are not specifying end index then it will consider up to end of the string.

The default value for step is 1.

Step value can be either +ve or –ve.

```
If +ve then it should be forward direction(left to right) and we ha
ve to consider begin to end- 1.
If -ve then it should be backward direction (right to left) and we
have to consider begin to end+1.
```

In [4]:
```
s="Learning Python is very very easy!!!"
print(s[1:7:1])
print(s[1:7])
print(s[1:7:2])
print(s[:7])
print(s[7:])
print(s[::])
print(s[:])
print(s[::-1])
```

```
earnin
earnin
eri
Learnin
g Python is very very easy!!!
Learning Python is very very easy!!!
Learning Python is very very easy!!!
!!!ysae yrev yrev si nohtyP gninraeL
```

In [12]:
```
print("1-",s[0:6:2])
print("2-",s[2:4:2])
print("3-",s[2::-1])
print("4-",s[:-1:3])
print("5-",s[::2])
print("6-",s[0:7:3])
print("7-",s[2:-1:2])
print("8-",s[:-2:2])
print("9-",s[1:-1:3])
print("10-",s[-1:-5:-2])#negative index starting number grater than ending
print("11-",s[-5::])
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-12-73824815cbd1> in <module>
----> 1 print("1-",s[0:6:2])
      2 print("2-",s[2:4:2])
      3 print("3-",s[2::-1])
      4 print("4-",s[:-1:3])
      5 print("5-",s[::2])

NameError: name 's' is not defined
```

In [13]:
```
s[0]='H' # we can not change string
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-13-8db904f6ee08> in <module>
----> 1 s[0]='H' # we can not change string

NameError: name 's' is not defined
```

In [14]:
```
s="hello world"
print(id(s))
del s
print(id(s))
```

```
3012818681136

---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-14-f05264e638e3> in <module>
      2 print(id(s))
      3 del s
----> 4 print(id(s))

NameError: name 's' is not defined
```

In [16]:
```
s="hello world"
del s[1]
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-16-be4f0deffe1e> in <module>
      1 s="hello world"
----> 2 del s[1]

TypeError: 'str' object doesn't support item deletion
```

Operations on Strings

Arithmetic Operations

Relational Operations

Logical Operations

Loops on Strings

Membership Operations

Arithmetic Operations----+, *

In [17]:
```
print("gandhinagar"+" "+"delhi")
```

```
gandhinagar delhi
```

In [18]:
```
print(5*" delhi")
```

```
 delhi delhi delhi delhi delhi
```

In [20]:
```
str="hello "
print(id(str))
str+="student"
print(str)
print(id(str))
```

```
3012818768176
hello student
3012818767984
```

Relational Operations

In [21]:
```
"delhi"=="gandhinagar"
```

Out[21]:
```
False
```

In [24]:
```
print("delhi"=="delhi")
print('delhi' != 'delhi')
print('mumbai' > 'pune')
```

```
True
False
False
```

Logical Operations

In [25]:
```
print('hello' and 'world')
print('hello' or 'world')
print('' and 'world')
print('' or 'world')
print(not 'world')
print(not '')
```

```
world
hello

world
False
True
```

Loops on Strings

In [26]:
```
str="hello "
index=0
for i in str:
    print('str [',index,"] = ",i)
    index+=1
```

```
str [ 0 ] =  h
str [ 1 ] =  e
str [ 2 ] =  l
str [ 3 ] =  l
str [ 4 ] =  o
str [ 5 ] =
```

In [27]:
```
str="hello "
for i in str:
    print("python")
```

```
python
python
python
python
python
python
```

In [28]:
```
str="hello "
for index,j in enumerate(str):
    print(index,"--",j)
```

```
0 -- h
1 -- e
2 -- l
3 -- l
4 -- o
5 --
```

Membership Operations

In [29]:
```
'D' in 'delhi'
```

Out[29]: False

In [30]:
```
"l" not in "mango"
```

Out[30]: True

Important Functions Of String:

len()

This len() method returns the length of the string. Syntax:len(str)

max()

The max() method returns the maximized alphabetical character from the string

Syntax:max(str)

min()

The min() method returns the minimum alphabetical character in the string str.

Syntax:min(str)

sorted()

The sorted() function returns a sorted list of the specified iterable object.

You can specify ascending or descending order. Strings are sorted alphabetically, and numbers are sorted numerically.

Note: You cannot sort a list that contains BOTH string values AND numeric values.

In [32]:
```
str="hello word"
print(len(str))#give total character of string
print(max(str))#give biggest character according to Ascii value
print(min(str))#give smallest character according to Ascii value here space
str="helloword"
print(min(str))
print(sorted(str))
print(sorted(str,reverse=True))
```

```
10
w

d
['d', 'e', 'h', 'l', 'l', 'o', 'o', 'r', 'w']
['w', 'r', 'o', 'o', 'l', 'l', 'h', 'e', 'd']
```

In [33]:
```
import string
dir(string)
```

Out[33]:
```
['Formatter',
 'Template',
 '_ChainMap',
 '_TemplateMetaclass',
 '__all__',
 '__builtins__',
 '__cached__',
 '__doc__',
 '__file__',
 '__loader__',
 '__name__',
 '__package__',
 '__spec__',
 '_re',
 '_sentinel_dict',
 '_string',
 'ascii_letters',
 'ascii_lowercase',
 'ascii_uppercase',
 'capwords',
 'digits',
 'hexdigits',
 'octdigits',
 'printable',
 'punctuation',
 'whitespace']
```

In [34]:
```
str="hello"
print(dir(str))
```

```
['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc_
_', '__eq__', '__format__', '__ge__', '__getattribute__', '__getitem__', '_
_getnewargs__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__i
ter__', '__le__', '__len__', '__lt__', '__mod__', '__mul__', '__ne__', '__n
ew__', '__reduce__', '__reduce_ex__', '__repr__', '__rmod__', '__rmul__',
'__setattr__', '__sizeof__', '__str__', '__subclasshook__', 'capitalize',
'casefold', 'center', 'count', 'encode', 'endswith', 'expandtabs', 'find',
'format', 'format_map', 'index', 'isalnum', 'isalpha', 'isascii', 'isdecima
l', 'isdigit', 'isidentifier', 'islower', 'isnumeric', 'isprintable', 'issp
ace', 'istitle', 'isupper', 'join', 'ljust', 'lower', 'lstrip', 'maketran
s', 'partition', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rspl
it', 'rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase', 't
itle', 'translate', 'upper', 'zfill']
```

In [40]:
```
s="learning Python is very very easy!!!"
print(s.capitalize())
print(s.title())
print(s.title())
print(s.lower())
print(s.upper())
print(s.swapcase())
print(s.count("i")) #count char
```

```
Learning python is very very easy!!!
Learning Python Is Very Very Easy!!!
Learning Python Is Very Very Easy!!!
learning python is very very easy!!!
LEARNING PYTHON IS VERY VERY EASY!!!
LEARNING pYTHON IS VERY VERY EASY!!!
2
```

String find() method

The find() method determines whether the given string str occurs in the string or in a substring of the string if the starting index beg and ending index end are given. Syntax: str.find(str, beg=0 end=len(string))

Parameters

str - It is the string needed to be found

beg - It is the starting index of the string; by default, it is zero

end - It indicates the ending index; by default, it is equal to the length of the string. Return type Index if found and -1 otherwise

In [44]:
```
s="A branch is an instruction in a computer program "
s.find("in")
```

Out[44]: 15

In [45]:
```
s.find("in",17,len(s))
```

Out[45]: 27

In [53]:
```
print(s.rfind("in"))
print(s.rfind("in",17,len(s)))
print(s.rfind("in",0,5))
```

```
27
27
-1
```

In [54]:
```
print(s.index("is"))
print(s.rindex("in"))
print(s.index("in",0,5))
```

```
9
27

---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-54-1ca1e91cf28d> in <module>
      1 print(s.index("is"))
      2 print(s.rindex("in"))
----> 3 print(s.index("in",0,5))

ValueError: substring not found
```

In [55]:
```
print(s.rindex("in",0,5))
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-55-60445f066885> in <module>
----> 1 print(s.rindex("in",0,5))

ValueError: substring not found
```

startswith()

In [56]:
```
'my name is nitish'.startswith('my')
```

Out[56]: True

In [57]:
```
'my name is nitish'.startswith('mmy')
```

Out[57]: False

endswith()

In [58]:
```
'my name is nitish'.endswith('sh')
```

Out[58]: True

isalnum()

In [61]:
```
s="a123bs"
a="abs"
print(s.isalnum())
print(a.isalnum())
```

```
True
True
```

In [62]:
```
s="a12  3bs"
print(s.isalnum())
```

```
False
```

isalpha() ,islower(),isupper(),isnumeric(),isdigit(),isspace(),istitle() ,isidentifier() ,join() ,replace(),split() ,lstrip() ,lstrip(),strip()

In [63]:
```
"12345698".isalpha()
```

Out[63]:
```
False
```

In [72]:
```
"abc".isalpha()
```

Out[72]:
```
True
```

In [65]:
```
"abcd".islower()
```

Out[65]:
```
True
```

In [66]:
```
"abcd".isupper()
```

Out[66]:
```
False
```

In [67]:
```
"123456".isnumeric()
```

Out[67]:
```
True
```

In [68]:
```
"123456".isdigit()
```

Out[68]:
```
True
```

In [70]:
```
"asd 123".isspace()
```

Out[70]:
```
False
```

In [71]:
```
" ".isspace()
```

Out[71]:
```
True
```

In [74]:
```
print("Python".istitle())
```

```
True
```

In [75]:
```
print("False".isidentifier())
```

```
True
```

In [76]:
```
" ".join(['hi', 'my', 'name', 'is', 'abc'])
```

Out[76]:
```
'hi my name is abc'
```

In [77]:
```
"swati ".join("patel")
```

Out[77]:
```
'pswati aswati tswati eswati l'
```

In [78]:
```
"<".join("123")
```

Out[78]:
```
'1<2<3'
```

In [79]:
```
"<".join(["123"])
```

Out[79]:
```
'123'
```

In [80]:
```
s="A branch is an instruction in a computer program."
s.split()
```

Out[80]:
```
['A', 'branch', 'is', 'an', 'instruction', 'in', 'a', 'computer', 'progra
m.']
```

In [81]:
```
s="A branch is an instruction in a computer program that can cause a compute
s.split("in")
```

Out[81]:
```
['A branch is an ',
 'struction ',
 ' a computer program that can cause a computer to beg',
 ' execut',
 'g a different ',
 'struction']
```

In [84]:
```
s="A branch is an instruction in a computer program that can cause a compute
print(s.replace("in","of"))
print(s.replace("in","of",2))#, only the first count occurrences are replace
```

```
A branch is an ofstruction of a computer program that can cause a computer
to begof executofg a different ofstruction
A branch is an ofstruction of a computer program that can cause a computer
to begin executing a different instruction
```

In [85]:
```
"abcswati patel".lstrip("abc")
```

Out[85]:
```
'swati patel'
```

In [86]:
```
"abcswati patel".rstrip("l")
```

Out[86]:
```
'abcswati pate'
```

In [88]:
```
"abcswati patelabc".strip("abc")
```

Out[88]:
```
'swati patel'
```

String maketrans()

Using each character in the intabstring is mapped into the character in the outtabstring at the same position.

Translate() then takes this table as input.

Syntax: str.maketrans(intab, outtab)

In [91]:
```
intab="aeiou"
outtab = "12345"
my = "this is string translate function…!!"
trantab =my.maketrans(intab, outtab)
print (my.translate(trantab))
```

```
th3s 3s str3ng tr1nsl1t2 f5nct34n…!!
```

In [92]:
```
import string
a_string = '!hi. wh?at is the weat[h]er lik?e.'
new_string = a_string.translate(str.maketrans('','',string.punctuation))
print(new_string)
```

```
hi what is the weather like
```

In [93]:
```
# Extract username from a given email.
# Eg if the email is nitish24singh@gmail.com # then the username should be n
s = input('enter the email')
pos = s.index('@')
print(s[0:pos])
```

```
enter the emailswatipatel@ljku.edu.in
swatipatel
```

In [97]:
```
# Count the frequency of a particular character in a provided string.
# Eg 'hello how are you' is the string, the frequency of h in this string is
s = input('enter the string  ')
term = input('what would like to search for  ')
counter = 0
for i in s:
    if i == term:
        counter += 1
print('frequency',counter)
```

```
enter the string  swati patel
what would like to search for  a
frequency 2
```

In [98]:
```
s = input('enter the string')
term = input('what would like to remove')
result = ''
for i in s:
    if i != term:
        result = result + i
print(result)
```

```
enter the stringswati h patel
what would like to removeh
swati  patel
```

In [104]:
```
# Write a program that can check whether a given string is palindrome or not
# abba
# malayalam
s = input('enter the string  ')
s1=s[::-1]
if s==s1:
    print('Palindrome')
else:
    print("not Pallindrom")
```

```
enter the string  asdf
not Pallindrom
```

In [109]:
```
# Write a program to count the number of words in a string without split()
s = input('enter the string  ')
temp = 1
for i in s:
    if i == ' ':
        temp = temp + 1
print("no. of words:",temp)
```

```
enter the string  swati h patel
no. of words: 3
```

In [111]:
```
# Write a python program to convert a string to title case without using the
s = input('enter the string')
L = []
for i in s.split():
    L.append(i[0].upper() + i[1:].lower())
print(" ".join(L))
```

```
enter the stringswati patel
Swati Patel
```

In [113]:
```
#Write a program to create a string made of first,middle and last character
s=input("enter string: ")
new_string=" "
new_string=s[0]+s[len(s)//2]+s[-1]
print(new_string)
```

```
enter string: swati
sai
```

In [115]:
```
#Write a program to find all occuences of a sub string in a given string by
s=input("enter string: ")
a=input("sub string: ")
print(s.upper().count((a.upper())))
```

```
enter string: swati
sub string: a
1
```

In [120]:
```
#Write a program to calculate the sum and average of the digits present in a
s=(input("enter string: "))
sum=0
c=0
for i in s:
    if i.isdigit()==True:
        sum+=int(i)
        c=c+1
print("sum",sum)
print(c)
print("avg=",sum/c)
```

```
enter string: s345p
sum 12
3
avg= 4.0
```

In [122]:
```
#Write a Python program to print even length words in a string.
s=input("enter string: ")
n=s.split(" ")
for i in n:
    if len(i)%2==0:
        print(i,end=" ")
```

```
enter string: asdf shp aaszxc
asdf aaszxc
```

In [124]:
```
#Write a Python program to Uppercase Half String from the given string.
s=input("enter string: ")
new_string1=s[0:len(s)//2:1].upper()
new_string2=s[len(s)//2:len(s)+1:1]
print(new_string1+new_string2)
```

```
enter string: swatip
SWAtip
```

In [126]:
```
#Write a Python program to capitalize the first and last character of each w
s=input("enter a string: ")
s = s.title()
result = ""
for word in s.split():
    result += word[:-1] + word[-1].upper() + " "
print(result)
```

```
enter a string: swati
SwatI
```

In [127]:
```
#Write a program to Create a string made of the middle three characters
s=input("enter string: ")
new_string=s[(len(s)//2)-1:(len(s)//2)+2]
print(new_string)
```

```
enter string: swati patel
i p
```

In [130]:
```
"""Write a program to check if two strings are balanced. For example,
strings s1 and s2 are balanced if all the characters in the s1 are present i
The character's position doesn't matter."""
s1=input("enter string: ")
s2= input("enter string: ")
flag=0
for i in s1:
    if i in s2:
        continue
    else:
        flag=1
if(flag ==1):
    print("not balanced")
else:
    print("balanced")
```

```
enter string: swati
enter string: pswatih
balanced
```

In [132]:
```
#Write a program to Split a string on hyphens
s1=input("enter string: ")
s2=s1.split("-")
print(s2)
```

```
enter string: swati-patel
['swati', 'patel']
```

In [135]:
```
#Replace each special symbol with # in the following string
import string
s=input("enter string: ")
for i in string.punctuation:
    s=s.replace(i,"#")
print(s)
```

```
enter string: swati@&*
swati###
```

In [138]:
```
"""Write program that takes user's name and pan card number
as input validate information using is x function and print the detail¶"""
pan_name=input("enter name: ")
if pan_name.isalpha()==False:
    print("sorry, wrong name")
pan_number= input("enter pan number: ")
if pan_number.isalnum()==False:
    print("sorry,wrong number")
print("correct")
```

```
enter name: s123
sorry, wrong name
enter pan number: 1234
correct
```

In [139]:
```
#Wpp program that encrypts a message by adding a key value to every characte
s="abcdefz "
index=0
while index<len(s):
    l=s[index]
    print(chr(ord(l)+3),end="")
    index+=1
```

```
defghi}#
```

In [140]:
```
#Wpp a program to generate abecedarian series
s="ABCDEFGH"
s1="ate"
for i in s:
    print(i+s1)
```

```
Aate
Bate
Cate
Date
Eate
Fate
Gate
Hate
```

In [142]:
```
#Wpp that accept a string from user and redisplay the same string after remo
s=input("enter string: ")
new=""
for i in s:
    if i in "aeiouAEIOU":
        continue
    else:
        new=new+i
print(new)
```

```
enter string: swati
swt
```

In [143]:
```
# Write a program  to remove i'th character from string in Python
test_str = "Hello welcome"
print ("The original string is : " + test_str)
new_str = ""
print("Enter index number which you want to scip")
j=int(input())
for i in range(len(test_str)):
    if i != j:
        new_str = new_str + test_str[i]
print ("The string after removal of i'th character : " + new_str)
```

```
The original string is : Hello welcome
Enter index number which you want to scip
4
The string after removal of i'th character : Hell welcome
```

Tuples

A tuple is a sequence of immutable or unchangeable Python objects, like lists.

The difference between tuples and lists is that tuples cannot be changed.

Additionally, lists use square brackets, whereas tuples use parentheses.

CHARACTERISTICS: Ordered .Unchangeble. Allows duplicate

Creating Tuple

Creating a tuple is as simple as separating values using commas.

You may also put these comma-separated values between parentheses.

In [8]:
```
# empty
t1 = ()
print(t1)
print(type(t1))
# create a tuple with a single item
t2 = ('hello',)
print(t2)
print(type(t2))
# homo
t3 = (1,2,3,4)
print(t3)
# hetro
t4 = (1,2.5,True,[1,2,3])
print(t4)
# tuple
t5 = (1,2,3,(4,5))
print(t5)
# using type conversion
t6 = tuple('hello')
print(t6)
t7 = ('hello')
print(t7)
print(type(t7))
print(type((1)))
print(type((1,)))
```

```
()
<class 'tuple'>
('hello',)
<class 'tuple'>
(1, 2, 3, 4)
(1, 2.5, True, [1, 2, 3])
(1, 2, 3, (4, 5))
('h', 'e', 'l', 'l', 'o')
hello
<class 'str'>
<class 'int'>
<class 'tuple'>
```

In [10]:
```
#ordered
(1,2,3)==(3,2,1)
```

Out[10]:
```
False
```

In [11]:
```
#unchangable
t=(1,2,3)
t[0]=4
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-11-43716500a615> in <module>
      1 #unchangable
      2 t=(1,2,3)
----> 3 t[0]=4

TypeError: 'tuple' object does not support item assignment
```

In [12]:
```
#duplicateS ALLOW
t=(3,1,2,1,3,2)
print(t)
```

```
(3, 1, 2, 1, 3, 2)
```

Accessing values in tuples

Use the square brackets to access values in a tuple and the index or indices to obtain a value of that index.

Indexing

Slicing

In [15]:
```
#indexing
t3 = (1,2,3,4)
print(t3)
print(t3[0])
print(t3[-1])
```

```
(1, 2, 3, 4)
1
4
```

In [16]:
```
#slicing
t3 = (1,2,3,4)
print(t3)
print(t3[0:4])
print(t3[-1:-4:-1])
```

```
(1, 2, 3, 4)
(1, 2, 3, 4)
(4, 3, 2)
```

Deleting tuple elements

Individual elements from a tuple cannot be removed, but there is nothing wrong with constructing another tuple without the unwanted elements.

In [17]:
```
t3=(1,2,3,4)
print(t3)
del t3
print(t3)
```

```
(1, 2, 3, 4)

---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-17-28493873aa9f> in <module>
      2 print(t3)
      3 del t3
----> 4 print(t3)

NameError: name 't3' is not defined
```

In [18]:
```
t3=(1,2,3,4)
print(t3)
del t3[2]
```

```
(1, 2, 3, 4)

---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-18-dee2eb729ce2> in <module>
      1 t3=(1,2,3,4)
      2 print(t3)
----> 3 del t3[2]

TypeError: 'tuple' object doesn't support item deletion
```

Operations on Tuples

In [21]:
```
# + and *
t1 = (1,2,3,4)
t2 = (5,6,7,8)
print(t1 + t2)
print(t1*3)
# membership
print(1 in t1)
# iteration
for i in t1:
    print(i)
for i,j in enumerate(t1):
    print(i,j)
```

```
(1, 2, 3, 4, 5, 6, 7, 8)
(1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4)
True
1
2
3
4
0 1
1 2
2 3
3 4
```

In [22]:
```
print((1,2,3)==(1,2,3,4))
print((1,2,"a")==(1,2,"A"))
print((1,2,"a")>(1,2,"A"))
print(("v",2,"a")>(1,2,"A"))
```

```
False
False
True
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-22-009919a568c1> in <module>
      2 print((1,2,"a")==(1,2,"A"))
      3 print((1,2,"a")>(1,2,"A"))
----> 4 print(("v",2,"a")>(1,2,"A"))

TypeError: '>' not supported between instances of 'str' and 'int'
```

Tuple in-built function

In [23]:
```
#len(tuple)
t=(1,2,3,4)
len(t)
```

Out[23]: 4

In [26]:
```
#Tuple max()
t=(1,2,3,4)
print(max(t))
t1=(1,2,3,4,"a")
print(max(t1))
```

```
4
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-26-f55f96b76dec> in <module>
      3 print(max(t))
      4 t1=(1,2,3,4,"a")
----> 5 print(max(t1))

TypeError: '>' not supported between instances of 'str' and 'int'
```

In [28]:
```
#Tuple min()
t=(1,2,3,4)
min(t)
```

Out[28]: 1

In [29]:
```
t=(1,2,3,4,'a')
min(t)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-29-f7e5dbb01b54> in <module>
      1 t=(1,2,3,4,'a')
----> 2 min(t)

TypeError: '<' not supported between instances of 'str' and 'int'
```

In [30]:
```
#Tuple tuple()
t=tuple(range(1,10))
print(t)
t=tuple([1,2,3,4])
print(t)
t=tuple("hello")
print(t)
```

```
(1, 2, 3, 4, 5, 6, 7, 8, 9)
(1, 2, 3, 4)
('h', 'e', 'l', 'l', 'o')
```

Tuple function

sorted ,count ,index ,sum

In [31]:
```
t=(1,5,2,8,3)
print(sorted(t))
print(sorted(t,reverse=True))
print(sorted(t,reverse=False))
```

```
[1, 2, 3, 5, 8]
[8, 5, 3, 2, 1]
[1, 2, 3, 5, 8]
```

In [32]:
```
t=(1,1,2,3,4,1,5,8)
t.count(1)
```

Out[32]: 3

In [33]:
```
t=(1,1,2,3,4,1,5,8)
t.index(1)
```

Out[33]: 0

In [40]:
```
t=(1,1,2,3,4,1,5,8)
t.index(1,3,7)# t.index(element,start index,end index)
#return index
```

Out[40]: 5

In [1]:
```
t=(1,1,2,3,4,1,5,8)
sum(t)
```

Out[1]:
```
25
```

In [3]:
```
# zipping tuples
a = (1,2,3,4)
b = (5,6,7,8)
tuple(zip(a,b))
```

Out[3]:
```
((1, 5), (2, 6), (3, 7), (4, 8))
```

In [5]:
```
#Check is tuples are same or not?
# write your code here
t1 = (1,2,3,0)
t2 = (1,2,3,0)

flag = True
for i,j in zip(t1,t2):
    if i == j:
        continue
    else:
        flag = False
        break
if flag:
    print('same')
else:
    print('not same')
```

```
same
```

In [6]:
```
#Write a program to print sum of even numbers and sum of odd numbers from el
test_tup = (1, 2, 3, 4, 5, 6)
odd=0
even=0
for i in test_tup:
    if (i%2==0):
        even+=i;
    else:
        odd+=i
print("Odd sum: ",odd)
print("Even sum: ",even)
```

```
Odd sum:  9
Even sum:  12
```

In [7]:
```
#Write a program to print Maximum and Minimum elements in given Tuple
test_tup = (1, 2, 3)
max=test_tup[0]
min=test_tup[0]
for i in test_tup:
    if (i>max):
        max=i
    if (i<min):
        min=i
print(max)
print(min)
```

```
3
1
```

In [10]:
```
#ord() and chr()
t=('A','B','C')
print(ord(t[0]))
t1=(97,98,99)
print(chr(t1[1]))
```

```
65
b
```

In [ ]: